

Optimization Algorithms in Neural–Network Training

(Ordered by Increasing Complexity)

The optimizers are presented from the *least* to the *most* sophisticated according to the amount of state they keep and the number of computations performed per iteration.

Common Notation

$\mathbf{x}_t$ Parameter (weight) vector at iteration t .

$f(\mathbf{x})$ Objective (loss) function to be minimised.

$\nabla_{\mathbf{x}}f(\mathbf{x})$ Gradient of f with respect to $\mathbf{x}$.

$\mathbf{g}_t$ Shorthand for $\nabla_{\mathbf{x}}f(\mathbf{x}_t)$.

α, η Base learning rate (step size).

γ Decay factor for the second–moment accumulator (RMSProp, backtracking).

β Momentum coefficient.

β_1, β_2 Exponential decay rates for first and second moments (Adam).

ε Small constant to prevent division by 0.

$\mathbf{v}_t$ Exponential moving average of past gradients (“momentum”).

$\mathbf{s}_t$ Exponential moving average of squared gradients (“variance”).

$\odot$ Hadamard (element–wise) product; $\mathbf{g}_t^{\odot 2}$ squares each coordinate.

1 Vanilla Gradient Descent (Fixed Step)

Update rule

$$\mathbf{g}_t = \nabla_{\mathbf{x}}f(\mathbf{x}_t), \quad \mathbf{d}_t = -\mathbf{g}_t \quad (1)$$

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \alpha \mathbf{d}_t \quad (2)$$

with constant step size α .

Rationale

A direct step opposite to the gradient. Simplicity is its strength, but the fixed α must be carefully tuned: too small slows convergence, too large causes divergence.

Python implementation

```
def gradient_descent(x0, f, grad, step_size,
                    niter=500, tol=1e-6):
    x = x0
    for t in range(niter):
        g = grad(x)
        if np.linalg.norm(g) <= tol:
            break
        x = x - step_size * g
    return x
```

2 Gradient Descent with Adaptive Step (Backtracking Line Search)

Update rule

1. $\mathbf{d}_t = -\nabla_x f(\mathbf{x}_t)$
2. Find α by backtracking:

while $f(\mathbf{x}_t + \alpha \mathbf{d}_t) > f(\mathbf{x}_t)$ set $\alpha \leftarrow \gamma \alpha$

3. $\mathbf{x}_{t+1} = \mathbf{x}_t + \alpha \mathbf{d}_t$

Rationale

Automatically shrinks the step until a decrease in the loss is observed, removing most of the learning-rate tuning burden while keeping the method conceptually simple.

Python implementation

```
def gradient_descent_adaptive_step(x0, f, grad, step_size,
                                   niter=500, tol=1e-6, gamma
                                   =0.98):
    x = x0
    for t in range(niter):
        g = grad(x)
        if np.linalg.norm(g) <= tol:
            break
        d = -g
        alpha = step_size
        while f(x + alpha * d) > f(x):
```

```
        alpha *= gamma
    x = x + alpha * d
    return x
```

3 Gradient Descent with Momentum

Update rule

$$\mathbf{g}_t = \nabla_x f(\mathbf{x}_t) \quad (3)$$

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \mathbf{g}_t \quad (4)$$

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \mathbf{v}_t \quad (5)$$

Rationale

The velocity $\mathbf{v}_t$ accumulates a decaying history of past gradients, acting like physical momentum. This accelerates learning along consistent directions and dampens oscillations across steep ravines.

Python implementation

```
def gradient_descent_momentum(x0, f, grad, niter=500, tol=1e-6,
                              eta=0.9, beta=0.98):
    x = x0
    v = np.zeros_like(x0)
    for t in range(niter):
        g = grad(x)
        if np.linalg.norm(g) <= tol:
            break
        v = beta * v + g
        x = x - eta * v
    return x
```

4 RMSProp

Update rule

$$\mathbf{g}_t = \nabla_x f(\mathbf{x}_t) \quad (6)$$

$$\mathbf{s}_t = \gamma \mathbf{s}_{t-1} + (1 - \gamma) \mathbf{g}_t^{\odot 2} \quad (7)$$

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \frac{\mathbf{g}_t}{\sqrt{\mathbf{s}_t} + \varepsilon} \quad (8)$$

Rationale

Keeps a moving average of squared gradients so each coordinate receives its own adaptive learning rate: large variances produce small effective steps and *vice versa*. Particularly effective in “ravine” landscapes with very different curvatures along different axes.

Python implementation

```
def rmsprop(x0, f, grad, niter=500, tol=1e-6,
            eta=0.9, gamma=0.98):
    epsilon = 1e-8
    x = x0
    s = np.zeros_like(x0)
    for t in range(niter):
        g = grad(x)
        if np.linalg.norm(g) <= tol:
            break
        s = gamma * s + (1 - gamma) * g**2
        x = x - eta * g / (np.sqrt(s) + epsilon)
    return x
```

5 Adam

Update rule

$$\mathbf{g}_t = \nabla_x f(\mathbf{x}_t) \quad (9)$$

$$\mathbf{v}_t = \beta_1 \mathbf{v}_{t-1} + (1 - \beta_1) \mathbf{g}_t \quad (10)$$

$$\mathbf{s}_t = \beta_2 \mathbf{s}_{t-1} + (1 - \beta_2) \mathbf{g}_t^{\odot 2} \quad (11)$$

$$\hat{\mathbf{v}}_t = \mathbf{v}_t / (1 - \beta_1^t), \quad \hat{\mathbf{s}}_t = \mathbf{s}_t / (1 - \beta_2^t) \quad (12)$$

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \frac{\hat{\mathbf{v}}_t}{\sqrt{\hat{\mathbf{s}}_t} + \varepsilon} \quad (13)$$

Rationale

Combines momentum (first moment) and per-coordinate variance scaling (second moment) while correcting early-iteration bias. Adam therefore adapts both direction and magnitude of updates, offering fast convergence with minimal hyperparameter tuning in practice.

Python implementation

```
def adam(x0, f, grad, niter=500, tol=1e-6,
         eta=0.9, beta1=0.9, beta2=0.99):
    epsilon = 1e-8
    x = x0
    v = np.zeros_like(x0)
    s = np.zeros_like(x0)
    t = 1
    while (t < niter) and (np.linalg.norm(grad(x)) > tol):
        g = grad(x)
        v = beta1 * v + (1 - beta1) * g
        s = beta2 * s + (1 - beta2) * g**2
        vt = v / (1 - beta1**t)
```

```
    st = s / (1 - beta2**t)
    x = x - eta * vt / (np.sqrt(st) + epsilon)
    t += 1
return x
```